

Merge Sort

Split index ranges, finish both child calls, and merge their sorted results using protected temporary output.

This complete offline chapter connects ideas, diagrams, exact traces, commented Java and exam practice. Switch themes for reading; printing always uses the light presentation and includes every required trace and answer.

GUIDE 2026-10-07.1 · STANDALONE HTML AND PDF SHARE THE SAME CONTENT

1. The array we will sort

Arrays use zero-based positions. Sorting changes which values occupy those positions. The target is ascending order; record tags would move with their values.

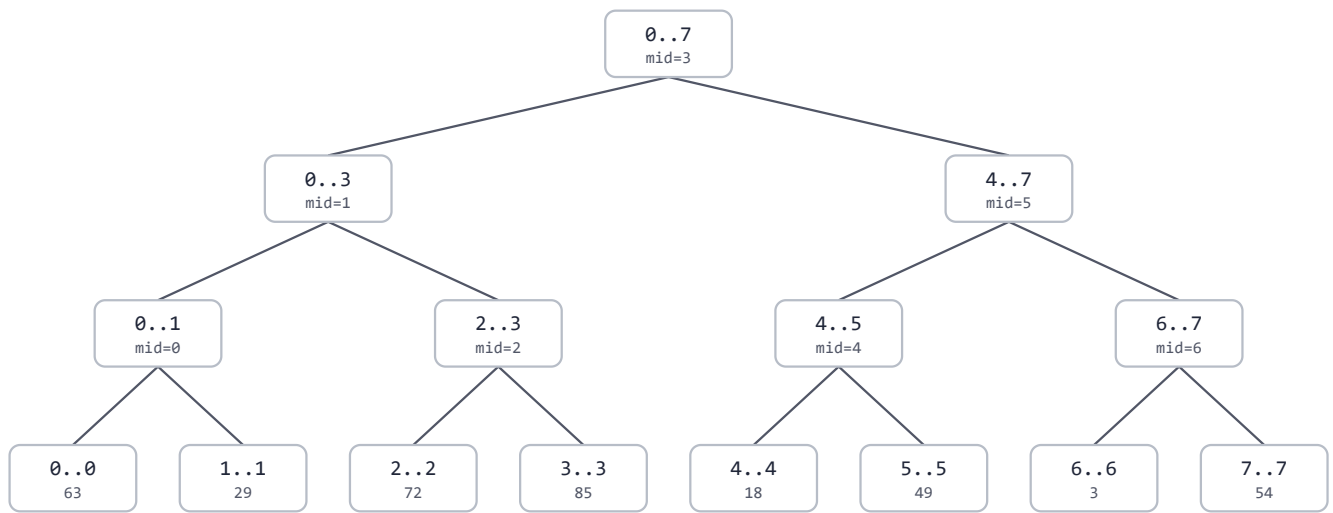
63	29	72	85	18	49	3	54
0	1	2	3	4	5	6	7

Initial input [63,29,72,85,18,49,3,54]. Inclusive range 0..7.

We will distinguish comparisons, swaps, shifts, temporary writes and copy-back writes. They are different units.

2. Definition and plain language

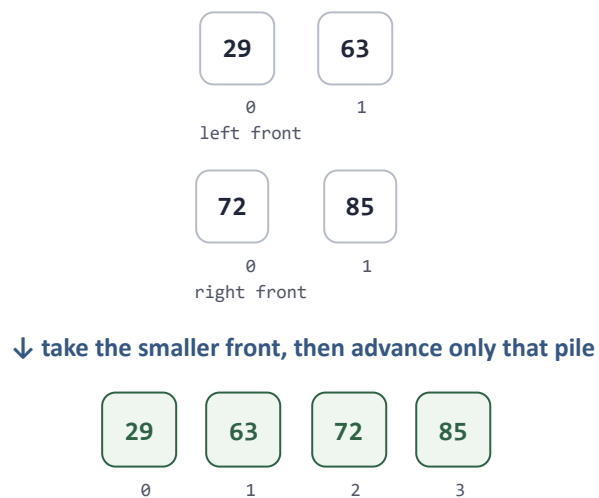
Merge sort is a divide-and-conquer comparison sort. It recursively sorts two disjoint halves and combines them by repeatedly selecting the smaller front value.



Divide the original problem into disjoint index ranges. One-value leaves are the base cases.

One card needs no sorting. Sort small piles first, then combine two sorted piles by taking their smallest remaining front card. Repeat until the original range is complete.

Two sorted piles



Merge works because both input runs are already sorted.

Splitting changes the problem boundaries, not the array values. Actual copying occurs during merging after both recursive child calls return.

3. Vocabulary and the central invariant

Term	Meaning in this chapter
start / end	Inclusive indexes bounding this call.
mid	Last index of the left half; the right begins at mid + 1.
Base case	start >= end: zero or one value.
left / right	First unread indexes in each sorted run.
out	Next free temp index, beginning at 0.
Stable merge	Choose left on equality using <=.

Invariant: Before a merge, both input runs are sorted. During merging, temp contains the smallest consumed values in order, and each pointer identifies the next unread value in its run.

4. Step through the complete execution

Predict the next decision before advancing. Next updates both the scene and the visibly marked trace row. Print includes the completed states and full table.

Depth-first calls and merge returns

▼ Full trace table - select a row to jump to its scene

Step	Operation	Reason	Shared array
1	Call 1: range 0..7	$\text{mid} = 0 + (7 - 0) / 2 = 3$. Values have not moved.	[63, 29, 72, 85, 18, 49, 3, 54]
2	Call 2: range 0..3	$\text{mid} = 0 + (3 - 0) / 2 = 1$. Values have not moved.	[63, 29, 72, 85, 18, 49, 3, 54]
3	Call 3: range 0..1	$\text{mid} = 0 + (1 - 0) / 2 = 0$. Values have not moved.	[63, 29, 72, 85, 18, 49, 3, 54]
4	Call 4: range 0..0	$0 \geq 0$: zero or one value, already sorted.	[63, 29, 72, 85, 18, 49, 3, 54]
5	Call 5: range 1..1	$1 \geq 1$: zero or one value, already sorted.	[63, 29, 72, 85, 18, 49, 3, 54]
6	Return 1: merge(0, 0, 1)	Return from call 3: [29, 63]. Both children completed before this merge.	[29, 63, 72, 85, 18, 49, 3, 54]
7	Call 6: range 2..3	$\text{mid} = 2 + (3 - 2) / 2 = 2$. Values have not moved.	[29, 63, 72, 85, 18, 49, 3, 54]

Step	Operation	Reason	Shared array
8	Call 7: range 2..2	$2 \geq 2$: zero or one value, already sorted.	[29, 63, 72, 85, 18, 49, 3, 54]
9	Call 8: range 3..3	$3 \geq 3$: zero or one value, already sorted.	[29, 63, 72, 85, 18, 49, 3, 54]
10	Return 2: merge(2, 2, 3)	Return from call 6: [72, 85]. Both children completed before this merge.	[29, 63, 72, 85, 18, 49, 3, 54]
11	Return 3: merge(0, 1, 3)	Return from call 2: [29, 63, 72, 85]. Both children completed before this merge.	[29, 63, 72, 85, 18, 49, 3, 54]
12	Call 9: range 4..7	$\text{mid} = 4 + (7 - 4) / 2 = 5$. Values have not moved.	[29, 63, 72, 85, 18, 49, 3, 54]
13	Call 10: range 4..5	$\text{mid} = 4 + (5 - 4) / 2 = 4$. Values have not moved.	[29, 63, 72, 85, 18, 49, 3, 54]
14	Call 11: range 4..4	$4 \geq 4$: zero or one value, already sorted.	[29, 63, 72, 85, 18, 49, 3, 54]
15	Call 12: range 5..5	$5 \geq 5$: zero or one value, already sorted.	[29, 63, 72, 85, 18, 49, 3, 54]
16	Return 4: merge(4, 4, 5)	Return from call 10: [18, 49]. Both children completed before this merge.	[29, 63, 72, 85, 18, 49, 3, 54]
17	Call 13: range 6..7	$\text{mid} = 6 + (7 - 6) / 2 = 6$. Values have not moved.	[29, 63, 72, 85, 18, 49, 3, 54]
18	Call 14: range 6..6	$6 \geq 6$: zero or one value, already sorted.	[29, 63, 72, 85, 18, 49, 3, 54]
19	Call 15: range 7..7	$7 \geq 7$: zero or one value, already sorted.	[29, 63, 72, 85, 18, 49, 3, 54]
20	Return 5: merge(6, 6, 7)	Return from call 13: [3, 54]. Both children completed before this merge.	[29, 63, 72, 85, 18, 49, 3, 54]
21	Return 6: merge(4, 5, 7)	Return from call 9: [3, 18, 49, 54]. Both children completed before this merge.	[29, 63, 72, 85, 3, 18, 49, 54]
22	Return 7: merge(0, 3, 7)	Return from call 1: [3, 18, 29, 49, 54, 63, 72, 85]. Both children completed before this merge.	[3, 18, 29, 49, 54, 63, 72, 85]

Final result: [3, 18, 29, 49, 54, 63, 72, 85].

5. Exam-style completed-pass / return table

One row per completed merge, in return order. Each row identifies its actual call and ranges.

Return	Call # / merge bounds	Left sorted run	Right sorted run	Merged result	Key comparisons
1	3 / (0, 0, 1)	[63]	[29]	[29,63]	1
2	6 / (2, 2, 3)	[72]	[85]	[72,85]	1
3	2 / (0, 1, 3)	[29,63]	[72,85]	[29,63,72,85]	2
4	10 / (4, 4, 5)	[18]	[49]	[18,49]	1
5	13 / (6, 6, 7)	[3]	[54]	[3,54]	1
6	9 / (4, 5, 7)	[18,49]	[3,54]	[3,18,49,54]	3
7	1 / (0, 3, 7)	[29,63,72,85]	[3,18,49,54]	[3,18,29,49,54,63,72,85]	5

6. What actually moves, and why

The completed-return view answers *which merge returns next?* This view answers *how does a merge construct its result?*

The final merge: every selection, leftover and copy-back

▼ Full trace table - select a row to jump to its scene

Step	left (absolute)	right (absolute)	out	Operation	temp
1	0	4	0	left = 0, right = 4, out = 0. Both input ranges are sorted.	[]
2	0	5	1	29 <= 3 → false; take right and advance only its pointer.	[3]
3	0	6	2	29 <= 18 → false; take right and advance only its pointer.	[3,18]
4	1	6	3	29 <= 49 → true; take left and advance only its pointer.	[3,18,29]
5	1	7	4	63 <= 49 → false; take right and advance only its pointer.	[3,18,29,49]
6	1	8	5	63 <= 54 → false; take right and advance only its pointer.	[3,18,29,49,54]
7	2	8	6	Right run is exhausted. Copy a remaining left value without a key comparison.	[3,18,29,49,54,63]
8	3	8	7	Right run is exhausted. Copy a remaining left value without a key comparison.	[3,18,29,49,54,63,72]
9	4	8	8	Right run is exhausted. Copy a remaining left value without a key comparison.	[3,18,29,49,54,63,72,85]
10	4	8	8	temp[0] is copied to arr[start + 0]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]

Step	left (absolute)	right (absolute)	out	Operation	temp
11	4	8	8	temp[1] is copied to arr[start + 1]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]
12	4	8	8	temp[2] is copied to arr[start + 2]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]
13	4	8	8	temp[3] is copied to arr[start + 3]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]
14	4	8	8	temp[4] is copied to arr[start + 4]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]
15	4	8	8	temp[5] is copied to arr[start + 5]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]
16	4	8	8	temp[6] is copied to arr[start + 6]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]
17	4	8	8	temp[7] is copied to arr[start + 7]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]

Start with two sorted runs

Left run 0..3; left = 0

29

63

72

85

0123

left

Right run 4..7; right = 4

3

18

49

54

4567

right

Temporary output; out = 0

.

.

.

.

.

.

.

.

01234567

Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced. Temporary output indexes begin at 0.

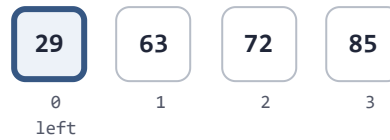
left = 0, right = 4, out = 0. Both input ranges are sorted.

CSI247 · Merge Sort

6 / 18

Write temp[1] = 18

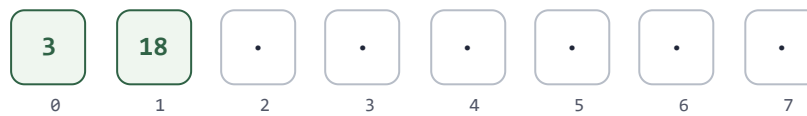
Left run 0..3; left = 0



Right run 4..7; right = 6



Temporary output; out = 2



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced. Temporary output indexes begin at 0.

29 <= 18 → false; take right and advance only its pointer.

Write temp[4] = 54

Left run 0..3; left = 1



Right run 4..7; right = 8



Temporary output; out = 5



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced. Temporary output indexes begin at 0.

63 <= 54 → false; take right and advance only its pointer.

Copy left leftover 85

Left run 0..3; left = 4



Right run 4..7; right = 8



Temporary output; out = 8



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced. Temporary output indexes begin at 0.

Right run is exhausted. Copy a remaining left value without a key comparison.

Copy back arr[7] = 85

Left run 0..3; left = 4



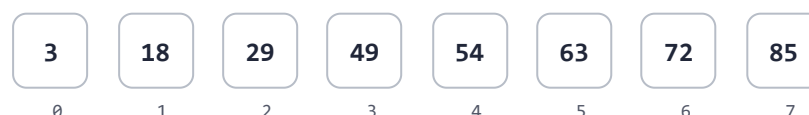
Right run 4..7; right = 8



Temporary output; out = 8



Shared array during copy-back



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced. Temporary output indexes begin at 0.

temp[7] is copied to arr[start + 7]. Unread source values were protected in temp.

Final merge: 5 key comparisons, 3 leftover copies, 8 temp writes, then 8 copy-back writes. Loop-bound checks are not counted as key comparisons.

The temporary cells protect every unread value. The output offset $\text{start} + k$ is essential when merging a subrange that does not begin at 0.

7. Fully commented Java implementation

JAVA

```
public static void mergeSort(int[] arr, int start, int end) {
    // Inclusive range of zero or one value: stop before splitting.
    if (start >= end) return;
    // Split INDEX RANGES. This statement does not move array values.
    int mid = start + (end - start) / 2;
    mergeSort(arr, start, mid);    // Finish the entire left subtree first.
    mergeSort(arr, mid + 1, end);  // Then finish the entire right subtree.
    // Both halves are now sorted: the merge precondition is satisfied.
    merge(arr, start, mid, end);
}

private static void merge(int[] arr, int start, int mid, int end) {
    // Protect unread input: never merge directly over unprotected values.
    int[] temp = new int[end - start + 1];
    int left = start;    // First unread value of left run start..mid.
    int right = mid + 1; // First unread value of right run mid+1..end.
    int out = 0;         // Next free position in temp (starts at zero).
    while (left <= mid && right <= end) {
        // Left-first ties preserve the order of equal records.
        if (arr[left] <= arr[right]) {
            temp[out] = arr[left]; left++;
        } else {
            temp[out] = arr[right]; right++;
        }
        out++; // Exactly one output value was written.
    }
    // One run is exhausted. Its partner's suffix is already sorted.
    while (left <= mid) { temp[out++] = arr[left++]; }
    while (right <= end) { temp[out++] = arr[right++]; }
    // Offset matters: temp[0] belongs at arr[start], not always arr[0].
    for (int k = 0; k < temp.length; k++) arr[start + k] = temp[k];
    // First call: mergeSort(arr, 0, arr.length - 1).
}
```

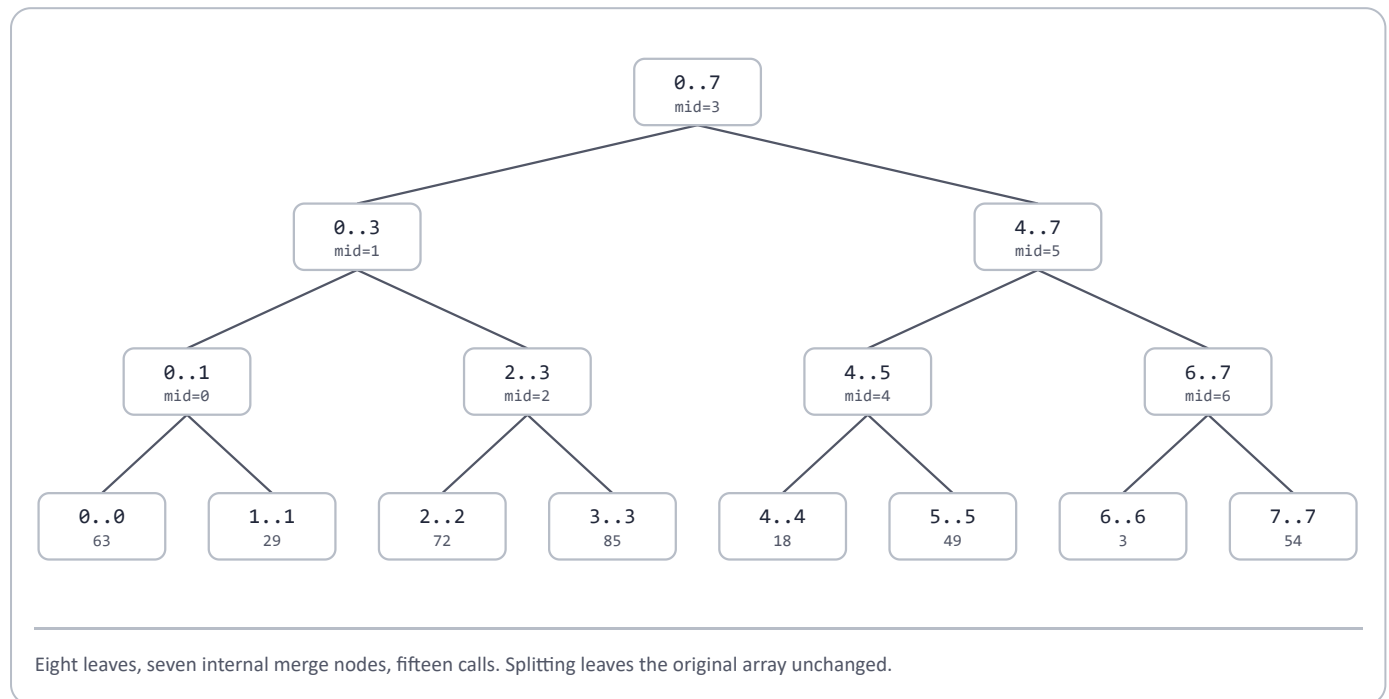
Write it from scratch

- State the inclusive base case.
- Calculate a midpoint that shrinks both ranges.
- Sort left, sort right, then merge.
- Track both input pointers and temp output.
- Choose one front value; advance only its source.
- Copy both possible leftover suffixes.

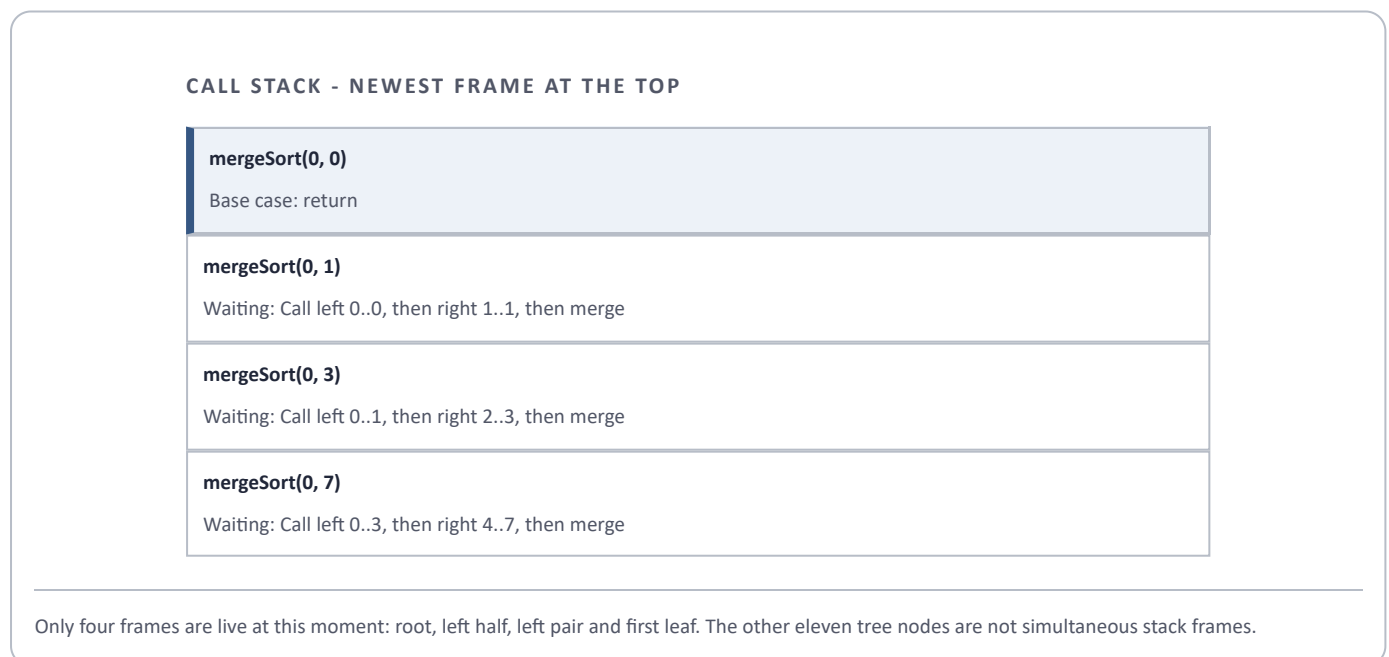
- Copy temp back with start + k.

8. Recursion, calls and returns

The primary implementation is already recursive. Its 15 calls form a tree, but Java executes them depth-first and retains only a path on the stack.



The live stack at the first base case



The base-case frame returns and disappears. Its parent then calls the right leaf, merges the pair, and returns to its own parent. A parent resumes exactly where its child call suspended it.

Every call, with its midpoint

Call #	Depth	Range	mid	Next work
1	0	0..7	3	left 0..3, right 4..7, then merge
2	1	0..3	1	left 0..1, right 2..3, then merge
3	2	0..1	0	left 0..0, right 1..1, then merge
4	3	0..0	-	Base case: return
5	3	1..1	-	Base case: return
6	2	2..3	2	left 2..2, right 3..3, then merge
7	3	2..2	-	Base case: return
8	3	3..3	-	Base case: return
9	1	4..7	5	left 4..5, right 6..7, then merge
10	2	4..5	4	left 4..4, right 5..5, then merge
11	3	4..4	-	Base case: return
12	3	5..5	-	Base case: return
13	2	6..7	6	left 6..6, right 7..7, then merge
14	3	6..6	-	Base case: return
15	3	7..7	-	Base case: return

For 0..3, calls visit 0..1 and its leaves, then merge 0..1, then visit 2..3 and its leaves, then merge 2..3, then merge 0..3. Merges are NOT executed level by level.

9. Complete runnable program

Save as MergeDemo.java. The code includes the method(s) and a main entry point.

JAVA

```
import java.util.Arrays;

public class MergeDemo {
    public static void mergeSort(int[] arr, int start, int end) {
        // Inclusive range of zero or one value: stop before splitting.
        if (start >= end) return;
        // Split INDEX RANGES. This statement does not move array values.
        int mid = start + (end - start) / 2;
        mergeSort(arr, start, mid);    // Finish the entire left subtree first.
        mergeSort(arr, mid + 1, end);  // Then finish the entire right subtree.
        // Both halves are now sorted: the merge precondition is satisfied.
        merge(arr, start, mid, end);
    }
}
```

```

private static void merge(int[] arr, int start, int mid, int end) {
    // Protect unread input: never merge directly over unprotected values.
    int[] temp = new int[end - start + 1];
    int left = start;    // First unread value of left run start..mid.
    int right = mid + 1; // First unread value of right run mid+1..end.
    int out = 0;         // Next free position in temp (starts at zero).
    while (left <= mid && right <= end) {
        // Left-first ties preserve the order of equal records.
        if (arr[left] <= arr[right]) {
            temp[out] = arr[left]; left++;
        } else {
            temp[out] = arr[right]; right++;
        }
        out++; // Exactly one output value was written.
    }
    // One run is exhausted. Its partner's suffix is already sorted.
    while (left <= mid) { temp[out++] = arr[left++]; }
    while (right <= end) { temp[out++] = arr[right++]; }
    // Offset matters: temp[0] belongs at arr[start], not always arr[0].
    for (int k = 0; k < temp.length; k++) arr[start + k] = temp[k];
    // First call: mergeSort(arr, 0, arr.length - 1).
}

public static void main(String[] args) {
    int[] values = {63, 29, 72, 85, 18, 49, 3, 54};
    mergeSort(values, 0, values.length - 1);
    System.out.println(Arrays.toString(values));
}

```

EXPECTED CONSOLE OUTPUT

```
[3, 18, 29, 49, 54, 63, 72, 85]
```

10. Descending order: change the decision

Use `arr[left] >= arr[right]` to choose the left run. Both children must also be sorted descending; left-first equality still preserves stability.

Keep the bounds and movement rules consistent. Reversing a final array is a different procedure and does not reproduce the descending trace of this implementation.

11. Five worked examples

Each case has input and result diagrams, evaluated decisions or merge inputs, exact counts, and a final state.

1. Nearly sorted

1

2

4

3

5

0

1

2

3

4

↓ ascending merge sort

1

2

3

4

5

0

1

2

3

4

The trace below connects the input diagram to the final result.

Return	Range	Inputs	Result	Key comparisons
1	0..1	[1] + [2]	[1,2]	1
2	0..2	[1,2] + [4]	[1,2,4]	2
3	3..4	[3] + [5]	[3,5]	1
4	0..4	[1,2,4] + [3,5]	[1,2,3,4,5]	4

8 key comparisons; 12 copy-back writes.

2. Reverse order

5

4

3

2

1

0

1

2

3

4

↓ ascending merge sort

1

2

3

4

5

0

1

2

3

4

The trace below connects the input diagram to the final result.

Return	Range	Inputs	Result	Key comparisons
1	0..1	[5] + [4]	[4,5]	1

Return	Range	Inputs	Result	Key comparisons
2	0..2	[4,5] + [3]	[3,4,5]	1
3	3..4	[2] + [1]	[1,2]	1
4	0..4	[3,4,5] + [1,2]	[1,2,3,4,5]	2

5 key comparisons; 12 copy-back writes.

3. Duplicate values

2

2

1

3

0

1

2

3

↓ ascending merge sort

1

2

2

3

0

1

2

3

The trace below connects the input diagram to the final result.

Return	Range	Inputs	Result	Key comparisons
1	0..1	[2] + [2]	[2,2]	1
2	2..3	[1] + [3]	[1,3]	1
3	0..3	[2,2] + [1,3]	[1,2,2,3]	3

5 key comparisons; 8 copy-back writes.

4. Uneven length

38

27

43

3

9

01234

↓ ascending merge sort

3

9

27

38

43

01234

The trace below connects the input diagram to the final result.

Return	Range	Inputs	Result	Key comparisons
1	0..1	[38] + [27]	[27,38]	1
2	0..2	[27,38] + [43]	[27,38,43]	2
3	3..4	[3] + [9]	[3,9]	1
4	0..4	[27,38,43] + [3,9]	[3,9,27,38,43]	2

6 key comparisons; 12 copy-back writes.

5. Descending target

8

3

5

1

0123

↓ descending merge sort

8

5

3

1

0123

The trace below connects the input diagram to the final result.

Return	Range	Inputs	Result	Key comparisons
1	0..1	[8] + [3]	[8,3]	1

Return	Range	Inputs	Result	Key comparisons
2	2..3	[5] + [1]	[5,1]	1
3	0..3	[8,3] + [5,1]	[8,5,3,1]	3

5 key comparisons; 8 copy-back writes. Use `arr[left] >= arr[right]` to choose the left run. Both children must also be sorted descending; left-first equality still preserves stability.

12. Properties and why the complexity occurs

Property	Result	Reason
Best / average / worst	$O(n \log n)$	The standard implementation still splits and merges sorted input.
Auxiliary arrays	$O(n)$ peak	The largest merge needs n temporary cells.
Call stack	$O(\log n)$	Only one root-to-leaf path is active at once.
Stable / in place / adaptive	Yes / no / no	Left-first ties preserve equals; the taught form allocates temp; no ordered-range shortcut.

8 values



1 run $\times$ 8 = 8

4 + 4



2 runs $\times$ 4 = 8

2 + 2 + 2 + 2



4 runs $\times$ 2 = 8

1-value leaves



Base cases: stop

Splitting gives $\log_2 n$ merge levels. Every merge level processes n values in total. The tree shows dependencies, not execution in level order.

For $n=8$, three merge levels each copy eight values, so this execution has 24 copy-back writes and 24 writes into temporary output. This is work across levels, not 24 simultaneous temporary cells: peak auxiliary storage remains $O(n)$. The active call path has $O(\log n)$ frames.

13. When to choose it

Suitable for large unpredictable arrays when consistent $O(n \log n)$ time and stability matter and temporary memory is available.

14. Common mistakes, symptoms and repairs

Attempt	What goes wrong	Repair
Right child starts at mid	Ranges overlap and may not shrink.	Use mid + 1.
Merge before both calls return	Input runs are not yet known to be sorted.	Call left, call right, then merge.
Ignore leftovers	Values vanish when one run becomes empty.	Copy the remaining suffix without key comparisons.
Copy temp[k] to arr[k]	A nonzero-start merge overwrites the wrong range.	Write arr[start + k].
Use < instead of <=	Equal right-run records can cross earlier left records.	Choose the left record on ties.

15. Exam-style practice and worked answers

A. Trace [8, 3, 5, 1] in descending order. Show every completed pass/merge. [8]

Return	Left	Right	Result
1	[8]	[3]	[8,3]
2	[5]	[1]	[5,1]
3	[8,3]	[5,1]	[8,5,3,1]

Final result [8,5,3,1]. Use $\text{arr}[\text{left}] \geq \text{arr}[\text{right}]$ to choose the left run. Both children must also be sorted descending; left-first equality still preserves stability.

B. Explain stability with tagged equal keys. [4]

On a tie between a left 2a and right 2b, $\leq$ selects 2a. Repeated left-first choices preserve original order.

C. Give the time and space, then justify both. [5]

Best / average / worst: $O(n \log n)$. The standard implementation still splits and merges sorted input. Auxiliary arrays: $O(n)$ peak. The largest merge needs n temporary cells. Call stack: $O(\log n)$. Only one root-to-leaf path is active at once. Stable / in place / adaptive: Yes / no / no. Left-first ties preserve equals; the taught form allocates temp; no ordered-range shortcut.

D. Debug the code before tracing it. [4]

Mistake	Symptom	Fix
Right child starts at mid	Ranges overlap and may not shrink.	Use mid + 1.
Merge before both calls return	Input runs are not yet known to be sorted.	Call left, call right, then merge.

E. What happens for [] and [7]? [2]

Both are already sorted. No key comparisons or value movements are required. Merge sort returns at its base case; the iterative methods never enter a pass.

F. Merge [2, 9] and [4, 5, 8]. Show pointers and leftovers. [6]

Choose 2 (left), 4 (right), 5 (right), 8 (right), then copy leftover 9. Four key comparisons, one leftover, five output writes. Relative pointers end at i=2,j=3; result [2,4,5,8,9].

16. Test-ready summary and mastery check

Split index ranges, finish both child calls, and merge their sorted results using protected temporary output.

- I can introduce every variable before using it.
- I can draw the current array with index labels.
- I can evaluate conditions with the actual values.
- I can distinguish pass summaries from individual operations.
- I can trace descending and duplicate cases.
- I can justify stability, time and auxiliary space.
- I can test empty, singleton, sorted and reverse input.